

Session 21

JavaScript Objects, this Keyword, Constructors, Prototypes, and Recursion

What are you going to learn in this session?

In this session, you will move beyond basic functions and begin exploring how JavaScript organizes and manages data using objects. Objects are one of the most important concepts in JavaScript because they allow developers to represent real-world entities such as users, products, orders, vehicles, and applications. You will learn how to create objects, access their properties, add new information dynamically, and remove unnecessary data when required.

You will also learn how JavaScript handles object creation through constructor functions and understand the purpose of the `this` keyword. These concepts help developers create reusable object templates and manage object-specific data efficiently. Additionally, you will explore prototypal inheritance, the mechanism JavaScript uses to share functionality between objects and reduce memory consumption.

Finally, this session introduces anonymous functions and recursion, two concepts that appear frequently in modern JavaScript development. Anonymous functions are commonly used in callbacks and event handling, while recursion helps solve complex problems by breaking them into smaller sub-problems. By the end of this session, you will be able to model real-world data using objects, create reusable object structures, understand inheritance concepts, and solve recursive programming challenges.

Why is this topic important?

Objects are at the heart of modern JavaScript applications. Almost everything in JavaScript can be represented as an object, including users, products, API responses, browser elements, and configuration settings. Popular frameworks and libraries such as React, Node.js, Express, and many backend systems heavily rely on object-based programming. Understanding objects is, therefore, a fundamental requirement for becoming a professional JavaScript developer.

Interviewers frequently ask questions about the `this` keyword, object manipulation, constructor functions, prototypes, and recursion because these concepts demonstrate a deeper understanding of JavaScript internals. Developers who understand object-oriented concepts write more maintainable code, design scalable applications, and collaborate more effectively within development teams. Mastering these topics also prepares learners for advanced concepts such as ES6 classes, design patterns, and framework-based development.

How should you prepare yourself to attend this session?

Before attending this session, you should be comfortable with JavaScript fundamentals, including variables, data types, operators, loops, conditions, and functions. Since constructor functions and recursion build directly upon function concepts, reviewing function declarations, parameters, return values, and scope will be especially helpful.

Ensure that your development environment is ready by having Node.js installed and VS Code configured properly. Practice creating simple functions and using variables inside functions. Reviewing object-like structures from other programming languages or real-life examples such as student records, employee details, and product catalogs will make object concepts easier to understand during the session.

What should you do after the session?

After completing this session, practice creating objects that represent real-world entities such as students, employees, books, products, and customers. Experiment with accessing, updating, adding, and deleting properties to understand how objects behave. Create constructor functions and generate multiple objects from them to strengthen your understanding of reusable object creation patterns.

You should also solve recursion exercises such as factorial calculation, countdown timers, and number summation problems. Build small projects like inventory systems, user profile managers, and product catalogs using objects and constructor functions. For interview preparation, practice explaining the `this` keyword, prototypal inheritance, and recursion using simple examples. Consistent practice will make these concepts easier to apply in real-world applications and technical interviews.

Class Notes

Introduction to Objects

An object is a collection of related data and functionality stored as key-value pairs.

Objects help represent real-world entities in code.

Example

JavaScript

```
const student = {  
  name: "Rahul",  
  age: 21,  
  course: "Computer Science"  
};  
  
console.log(student);
```

Why Objects Matter

Instead of storing related information in separate variables:

JavaScript

```
let name = "Rahul";  
let age = 21;  
let course = "Computer Science";
```

We can group them together:

JavaScript

```
const student = {  
  name: "Rahul",  
  age: 21,  
  course: "Computer Science"  
};
```

Real-World Examples

- User profiles
- Products in e-commerce
- Employee records

- API responses
- Banking accounts

Property Access

Object properties can be accessed using two methods.

Dot Notation

JavaScript

```
const user = {  
  name: "Amit",  
  age: 25  
};  
  
console.log(user.name);
```

Output:

None

Amit

Bracket Notation

JavaScript

```
console.log(user["age"]);
```

Output:

None

25

When to Use Bracket Notation

Useful when property names are dynamic.

JavaScript

```
const property = "name";

console.log(user[property]);
```

Best Practice

Use dot notation whenever possible because it is simpler and more readable.

Adding and Removing Properties

JavaScript objects are dynamic and can be modified after creation.

Adding Properties

JavaScript

```
const user = {
  name: "Priya"
};

user.city = "Bangalore";

console.log(user);
```

Output:

JavaScript

```
{
  name: "Priya",
  city: "Bangalore"
}
```

Updating Properties

JavaScript

```
user.city = "Mumbai";
```

Removing Properties

JavaScript

```
delete user.city;
```

Real-World Use Cases

- Updating user profiles
- Managing shopping carts
- Modifying application settings

The `this` Keyword

The `this` keyword refers to the object that is currently executing the method.

Example

JavaScript

```
const person = {  
  name: "Ankit",  
  
  greet() {  
    console.log("Hello " + this.name);  
  }  
};  
  
person.greet();
```

Output:

None

```
Hello Ankit
```

Why Use `this`?

Without `this`:

JavaScript

```
const person = {  
  name: "Ankit",  
  
  greet() {  
    console.log(name);  
  }  
};
```

This causes an error because the `name` is not found in the current scope.

Real-World Use Cases

- User objects
- Product objects
- Class methods
- Object-oriented programming

Constructor Functions

Constructor functions allow creation of multiple similar objects.

Example

JavaScript

```
function User(name, age) {  
  this.name = name;  
  this.age = age;  
}
```

Creating objects:

JavaScript

```
const user1 = new User("Rahul", 21);  
const user2 = new User("Priya", 22);  
  
console.log(user1);
```

```
console.log(user2);
```

Output:

JavaScript

```
User { name: "Rahul", age: 21 }  
User { name: "Priya", age: 22 }
```

Benefits

- Reusable object creation
- Cleaner code
- Consistent structure

Real-World Examples

- Customer records
- Product creation
- Student management systems

Prototypal Inheritance Concept

JavaScript uses prototypes to share methods and properties between objects.

Instead of creating duplicate methods for every object, methods can be shared through prototypes.

Example

JavaScript

```
function User(name) {  
  this.name = name;  
}  
  
User.prototype.greet = function () {  
  console.log("Hello " + this.name);  
};
```



```
const user1 = new User("Rahul");
const user2 = new User("Priya");

user1.greet();
user2.greet();
```

Benefits

- Reduces memory usage
- Promotes code reuse
- Improves performance

Concept Visualization

```
None
user1
  |
Prototype
  |
User.prototype
```

When JavaScript cannot find a property directly on an object, it searches the prototype chain.

Anonymous Functions

An anonymous function is a function without a name.

Example

```
JavaScript
const greet = function () {
  console.log("Hello");
};
```

The function itself has no name but is assigned to a variable.

Callback Example

JavaScript

```
setTimeout(function () {  
  console.log("Executed");  
}, 1000);
```

Common Uses

- Event handlers
- Callbacks
- Array methods
- Asynchronous programming

Recursion

Recursion is a technique where a function calls itself to solve a problem.

Every recursive function requires:

1. Base Case
2. Recursive Call

Example: Countdown

JavaScript

```
function countdown(number) {  
  if (number === 0) {  
    return;  
  }  
  
  console.log(number);  
  
  countdown(number - 1);  
}  
  
countdown(5);
```

Output:

```
None
5
4
3
2
1
```

Example: Factorial

```
JavaScript
function factorial(n) {
  if (n === 1) {
    return 1;
  }

  return n * factorial(n - 1);
}

console.log(factorial(5));
```

Output:

```
None
120
```

Advantages

- Elegant solution for certain problems
- Useful for tree traversal
- Useful for nested structures

Caution

A missing base case causes infinite recursion.

Interview Prep Recap

Students should now be able to confidently explain:

- What objects are and why they are useful
- Property access techniques
- Dynamic object modification
- Purpose of the `this` keyword
- Constructor function workflow
- Prototype-based inheritance
- Anonymous functions and callback usage
- Recursion and recursive thinking
- Object-oriented programming foundations

Sample Interview Questions

Q1. What is an object in JavaScript?

Answer:

An object is a collection of key-value pairs used to store related data and functionality.

Q2. What is the difference between dot notation and bracket notation?

Answer:

Dot notation uses fixed property names, while bracket notation supports dynamic property access.

Q3. What does `this` refer to?

Answer:

It refers to the object currently executing the method.

Q4. Why use constructor functions?

Answer:

They create multiple objects with the same structure efficiently.

Q5. What is a prototype?

Answer:

A prototype is an object that provides shared properties and methods to other objects.

Q6. What is recursion?

Answer:

A programming technique where a function calls itself until a base condition is met.

Q7. What is an anonymous function?

Answer:

A function without a name, often used as callbacks or assigned to variables.

Common Beginner Confusions

Confusion:1

Objects and arrays are the same thing.

Reality

Arrays store ordered values, while objects store key-value pairs.

How to Remember It

Array = List, Object = Dictionary.

Confusion:2

Dot notation and bracket notation are identical.

Reality

Bracket notation supports dynamic property names.

How to Remember It

Bracket = Flexible, Dot = Direct.

Confusion:3

`this` always refers to the current function.

Reality

`this` depends on how the function is called.

How to Remember It

`this` follows the caller, not the creator.

Confusion:4

Constructor functions are normal functions.

Reality

They are designed to create object instances using `new`.

How to Remember It

Constructor = Object factory.

Confusion:5

Every object gets its own copy of prototype methods.

Reality

Prototype methods are shared among all instances.

How to Remember It

Prototype = Shared toolbox.

Confusion:6

Anonymous functions cannot be used later.

Reality

They can be assigned to variables and reused.

How to Remember It

Unnamed does not mean unusable.

Confusion:7

Recursion is completely different from loops.

Reality

Both perform repetition; recursion uses function calls.

How to Remember It

Loop repeats with iteration, recursion repeats with function calls.

Confusion:8

Recursion always runs forever.

Reality

A proper base case stops recursion.

How to Remember It

Every recursive function needs an exit door.

Confusion:9

Deleting a property deletes the entire object.

Reality

Only the specified property is removed.

How to Remember It

Delete key, not container.

Key Takeaways

In this session, you learned how JavaScript uses objects to represent and organize real-world data. You explored object creation, property access techniques, and dynamic modification of object properties. You also gained an understanding of the `this` keyword and how it helps

methods interact with their owning objects. These concepts form the basis of object-oriented programming in JavaScript.

You learned how constructor functions create reusable object templates and how prototypal inheritance enables objects to share functionality efficiently. These concepts explain how JavaScript manages object relationships internally and prepare you for learning modern ES6 classes and advanced design patterns.

Additionally, you explored anonymous functions and recursion, both of which are frequently used in professional JavaScript development. Recursion strengthens problem-solving skills by teaching developers to think in smaller, repeatable steps, while anonymous functions are heavily used in callbacks, events, and asynchronous operations. Mastering these topics builds a strong foundation for advanced JavaScript, framework development, and technical interview success.